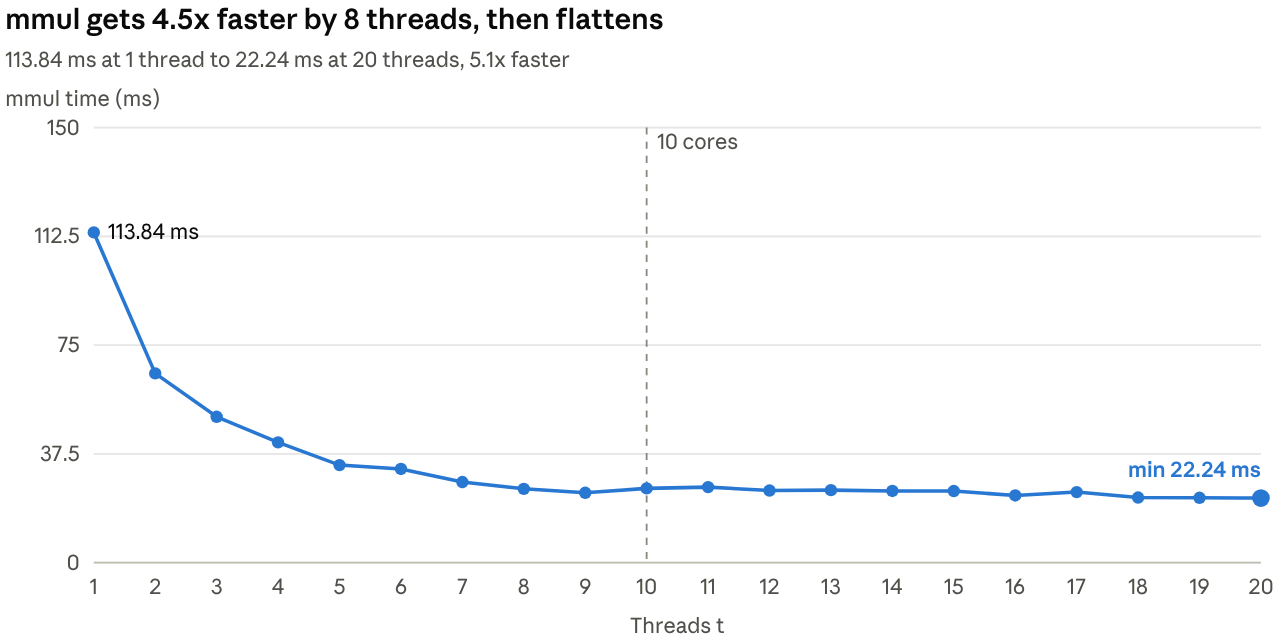
# ECE 759 Assignment 3

Oct 16, 2026 · @Praneet Popuri

GitHub link: https://github.com/praneetpopuri1/repo759 (HW03 folder)

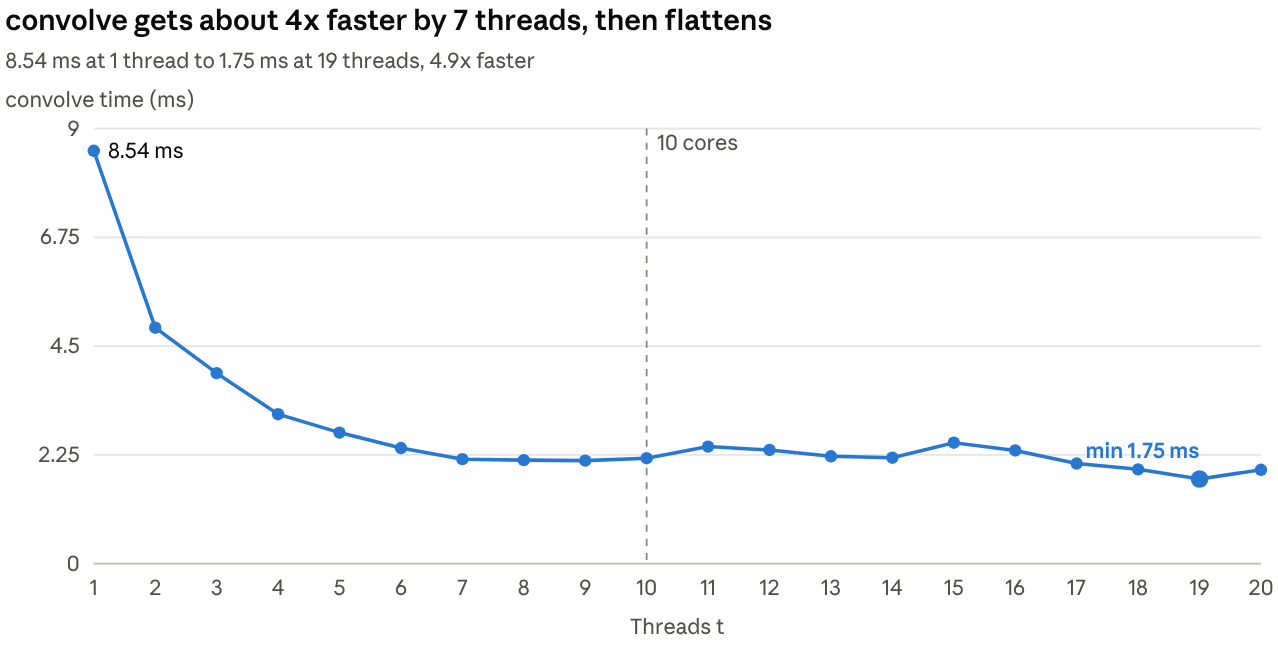
## Problem 1: mmul time vs. threads (n = 1024)




task1.out · median of 5 runs per point · n = 1024

Time drops steeply up to about 8 threads, then levels off at 22-25 ms through t = 20.

# Problem 2: convolve time vs. threads (n = 1024)




task2.out · median of 5 runs per point · n = 1024, 3x3 mask

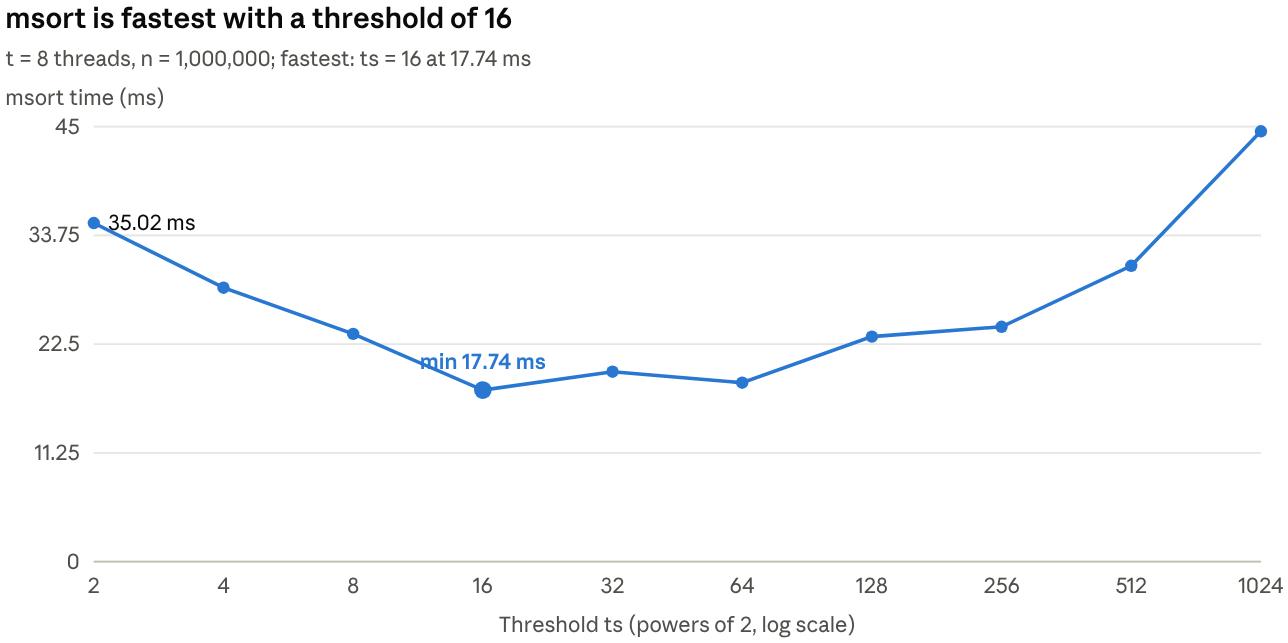
More threads help almost linearly at first: 2 and 4 threads come close to the ideal 2x and 4x speedups. From about t = 7 to t = 20 the curve is essentially flat; the lowest point at t = 19 is within run-to-run noise of its neighbours.

Why the run time stops dropping:

1. **Limited cores.** With schedule(static) every thread gets the same share of rows, so the slowest core decides when the loop ends. Once every core has a thread, extra threads just time-share them.
2. **Memory bandwidth.** A 3x3 convolution does only about 18 flops per output element but reads from a 4 MB image and writes a 4 MB output. Once a few threads run, the cores wait on memory rather than compute, and memory bandwidth does not grow with thread count.
3. **Fixed parallel overhead.** With several threads the whole kernel takes only about 2 ms. Waking the thread team, splitting the iterations and the closing barrier are a fixed cost, which becomes a larger share of a shrinking run time (Amdahl's law).

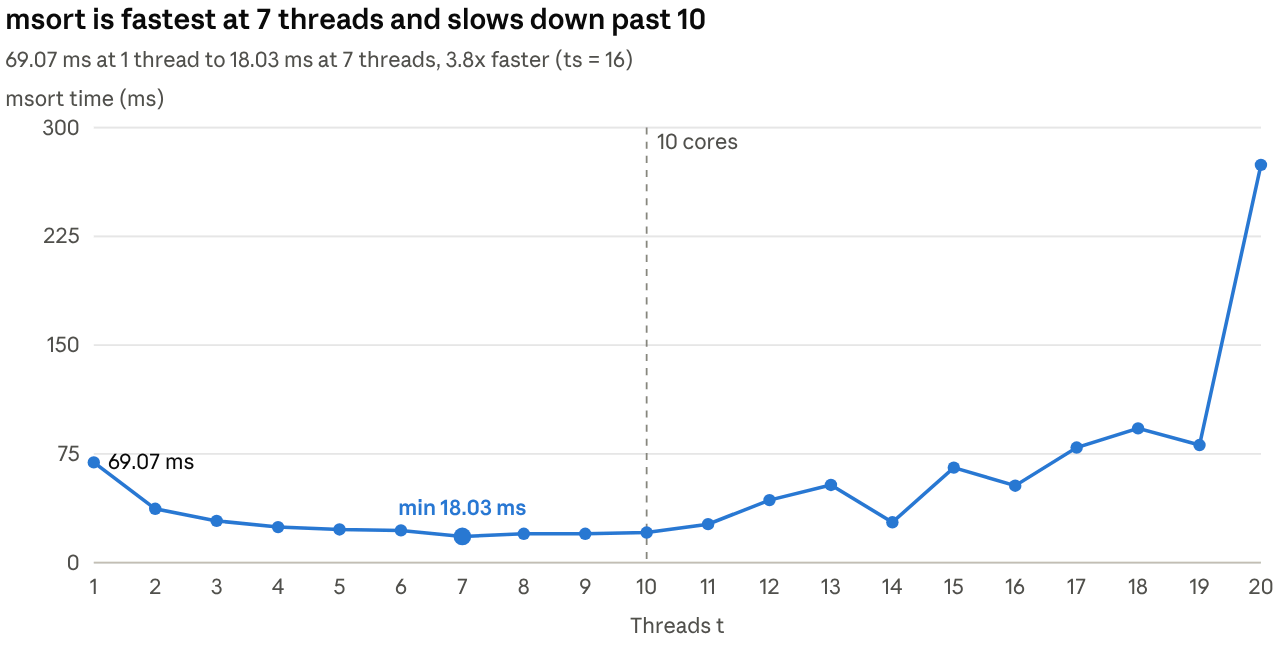
Together, more threads help until the cores or the memory system are saturated, about 7-8 threads here. After that the run time stays flat.

# Problem 3: msort time vs. threshold and threads




task3_ts.out · median of 5 runs per point · n = 1,000,000, t = 8

Very small thresholds create a huge number of tiny tasks, so task-creation overhead dominates. Large thresholds spend too long in the O(ts^2) insertion sort at the leaves. The best value, ts = 16, is used for the thread sweep below.




task3_t.out · median of 5 runs per point · n = 1,000,000, ts = 16

msort speeds up through 7-10 threads. Once threads outnumber cores it slows sharply, because the extra OpenMP threads spin-wait at taskwait while competing for the same cores.